GUÍA DE LABORATORIO: INTEGRACIÓN NUMÉRICA

Física Computacional I — Regla del Trapecio y Simpson en Sistemas Físicos Realistas

Estudiante: Abiud Mahiam Villamizar Ortiz **Fecha:** 19/10/2026

Grupo / Sección: Física Computacional **Calificación:** _____

1. OBJETIVOS DE LA PRÁCTICA

- Aplicar las reglas de integración del Trapecio Compuesto y Simpson 1/3 Compuesto a problemas de la mecánica clásica.
- Calcular analíticamente las cotas superiores de error numérico y determinar el tamaño de partición n requerido.
- Programar los algoritmos en Python y verificar experimentalmente los órdenes de convergencia $\mathcal{O}(h^2)$ y $\mathcal{O}(h^4)$.

2. MARCO TEÓRICO Y PROBLEMA FÍSICO

Para un péndulo simple de longitud L, la aproximación de pequeñas oscilaciones da un período $T_0 = 2\pi\sqrt{(L/g)}$. Para amplitudes elevadas θ_0, el período exacto viene dado por la integral elíptica completa de primera especie:

$$T(\theta_0) = 4\sqrt{(L/g)}\int_0^{\pi/2} [\ 1 / \sqrt{(1 - k^2\sin^2\varphi)}\ ]\, d\varphi\ , \qquad \text{donde} \quad k = \sin(\theta_0 / 2)$$

3. PARTE 1: ANÁLISIS ANALÍTICO DEL ERROR

Parámetros del sistema: L = 1.0 m, g = 9.81 m/s^2, amplitud inicial θ_0 = 60° (π/3 rad).

Tarea 1.1: Escribe los parámetros de la integral a resolver:
• Parámetro k = sin(30°) = 0,5
• Valor del período aproximado $T_0 = 2\pi\sqrt{(1.0 / 9.81)}$ = 2,006066 segundos.

Sabiendo que para $f(\varphi) = (1 - 0.25\ sin^2\varphi)^{-1/2}$ en el intervalo [0, π/2], las cotas superiores de las derivadas son $M_2 = max\ |f''(\varphi)| \le 0.75$ y $M_4 = max\ |f^{(4)}(\varphi)| \le 3.5$:

Método Numérico	Cota Teórica del Error	Cálculo Mínimo de n para Error < 10^{-4}
Trapecio Compuesto	$\lvert E_T\rvert \le [\ (b-a)^3 / (12\ n^2)\ ]\ M_2$	n mínimo = 50
Simpson 1/3 Compuesto	$\lvert E_S\rvert \le [\ (b-a)^5 / (180\ n^4)\ ]\ M_4$	n mínimo (par) = 8

4. PARTE 2: IMPLEMENTACIÓN DE ALGORITMOS EN PYTHON

Completa el código fuente para implementar las funciones de integración numérica utilizando vectorización de NumPy.

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import quad

# --- Parámetros Físicos ---
L = 1.0
g = 9.81
theta_0 = np.radians(60)
k = np.sin(theta_0 / 2.0)

def integrando(phi):
    """Función f(phi) para la integral elíptica del péndulo."""
    return 1.0 / np.sqrt(1.0 - (k**2) * (np.sin(phi)**2))

def trapecio_compuesto(f, a, b, n):
    """Calcula la integral usando la Regla del Trapecio Compuesto."""
    h = (b - a) / n
    x = np.linspace(a, b, n + 1)
    y = f(x)
    # COMPLETAR: Aplica la fórmula del Trapecio
    integral = (h / 2.0) * (y[0] + 2 * np.sum(y[1:-1]) + y[-1])
    return integral

def simpson_13_compuesto(f, a, b, n):
    """Calcula la integral usando Simpson 1/3 (n debe ser par)."""
    if n % 2 != 0:
        raise ValueError("El número de subintervalos n debe ser PAR.")
    h = (b - a) / n
    x = np.linspace(a, b, n + 1)
    y = f(x)
    # COMPLETAR: Suma de índices impares y pares
    suma_impares = np.sum(y[1:-1:2])
    suma_pares = np.sum(y[2:-2:2])
    integral = (h / 3.0) * (y[0] + 4 * suma_impares + 2 * suma_pares + y[-1])
    return integral
```

5. PARTE 3: ANÁLISIS DE CONVERGENCIA DEL ERROR

Escribe la rutina para evaluar el error absoluto frente al valor exacto devuelto por `scipy.integrate.quad` para valores de $n = [2, 4, 8, 16, 32, 64, 128, 256]$.

```python
# Valor de referencia con alta precisión
I_exacto, _ = quad(integrando, 0, np.pi/2)

n_valores = np.array([2, 4, 8, 16, 32, 64, 128, 256])
err_trap = []
err_simp = []

for n in n_valores:
    I_t = trapecio_compuesto(integrando, 0, np.pi/2, n)
    I_s = simpson_13_compuesto(integrando, 0, np.pi/2, n)
    err_trap.append(abs(I_t - I_exacto))
    err_simp.append(abs(I_s - I_exacto))

# Gráfica Log-Log
plt.figure(figsize=(7, 4))
```

6. RESULTADOS NUMÉRICOS OBTENIDOS

Registra los valores computados en la ejecución con *n = 100* subintervalos:

Concepto	Valor Calculado	Unidades / Observación
Integral *I* (Trapecio, n=100)	1.68575035	Adimensional
Integral *I* (Simpson 1/3, n=100)	1.68575035	Adimensional
Período Real *T(60°)*	2.152876	Segundos (s)
Error Relativo vs T_0	7.32%	% de subestimación del modelo lineal

7. PREGUNTAS DE DISCUSIÓN Y CONCLUSIONES

Pregunta 1: Análisis de las Pendientes en Log-Log

A partir del gráfico generado, explica cómo se relacionan la pendiente visual de cada curva con el orden teórico del error $O(h^2)$ y $O(h^4)$.

En una gráfica Log-Log, la pendiente de cada curva permite identificar el orden de convergencia del error numérico. En este caso, el método del Trapecio (curva azul) presenta un error de orden $(O(h^2)$, mientras que el método de Simpson (curva naranja) presenta un error de orden $(O(h^4)$.Esto significa que, al disminuir el tamaño del paso (h), el error de Simpson se reduce más rápidamente que el del Trapecio. Por lo tanto, una pendiente más pronunciada hacia abajo indica una mayor velocidad de convergencia y una reducción más rápida del error.

Pregunta 2: Comparación de Eficiencia Computacional

Para alcanzar una tolerancia de 10^{-8}, ¿cuántos intervalos requiere aproximadamente cada método? ¿Cuál es más eficiente?

Al comparar ambos métodos en la gráfica, se observa que el método del Trapecio alcanza el error absoluto de 10^{-8}con un número menor de intervalos, mientras que Simpson alcanza ese nivel aproximadamente cuando $n \approx 10$, según los valores indicados en la gráfica. Sin embargo, es importante considerar que la eficiencia no depende únicamente del error alcanzado inicialmente, sino también de la rapidez con la que este disminuye al aumentar el número de intervalos. Debido a su orden de convergencia $O(h^4)$, el método de Simpson reduce el error más rápidamente que el método del Trapecio, cuyo orden es $O(h^2)$. Por esta razón, Simpson suele ser más eficiente cuando se requieren tolerancias de error muy pequeñas, aunque el resultado concreto depende de la función evaluada y de los valores obtenidos.

Pregunta 3: Límite Numérico y Precisión de Punto Flotante

Si incrementas *n* a 10^8, ¿el error disminuirá indefinidamente o comenzará a aumentar? Explica la competencia entre el error de truncamiento y el error de redondeo (IEEE 754).

La gráfica muestra que el error no disminuye indefinidamente al aumentar el número de intervalos. A partir de aproximadamente $n \approx 16$, ambas curvas tienden a estabilizarse alrededor de 10^{-15}, formando una zona en la que la reducción del error es mínima. Este comportamiento se debe a las limitaciones de precisión de la aritmética de punto flotante utilizada por la computadora. Cuando el error de truncamiento se vuelve suficientemente pequeño, los errores de redondeo comienzan a tener una mayor influencia en el resultado. Por ello, aumentar excesivamente el número de intervalos no garantiza una mayor precisión e incluso puede provocar un ligero incremento del error. Este fenómeno evidencia la existencia de un límite práctico de precisión numérica asociado a la representación de números reales en la computadora.

Firma del Estudiante: Abiud Villamizar

Fecha de Entrega: 19 / 10 / 2026